Practical 10: MiniBank Concurrent Transaction Processor Engine

Problem Definition:

Build MiniBank's high-throughput TransactionProcessor utilizing an ExecutorService thread pool, submitting batch transactions, and executing thread-safe inter-account transfers with deadlock prevention via account number lock ordering.

Code:

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

class PartB {
    public static class Account {
        private final String accountNumber;
        private final String holderName;
        private long balance;

        public Account(String accountNumber, String holderName, long balance) {
            this.accountNumber = accountNumber;
            this.holderName = holderName;
            this.balance = balance;
        }

        public String getAccountNumber() { return accountNumber; }
        public String getHolderName() { return holderName; }
        public synchronized long getBalance() { return balance; }

        public synchronized void deposit(long amount) {
            balance += amount;
        }

        public synchronized boolean withdraw(long amount) {
            if (balance >= amount) {
                balance -= amount;
                return true;
            }
            return false;
        }

        @Override
        public String toString() {
            return "[" + accountNumber + " | " + holderName + " | Balance: Rs. " + getBalance()
+ "]";
        }
    }

    public static class TransactionProcessor {
        private final ExecutorService executor;

        public TransactionProcessor(int poolSize) {
            this.executor = Executors.newFixedThreadPool(poolSize);
        }

        public void submit(Runnable task) {
```

```
            executor.submit(task);
        }

        public void stop() {
            executor.shutdown();
            try {
                if (!executor.awaitTermination(5, TimeUnit.SECONDS)) {
                    executor.shutdownNow();
                }
            } catch (InterruptedException e) {
                executor.shutdownNow();
            }
        }

        public static void transferDeadlockFree(Account from, Account to, long amount) {
            Account first = (from.getAccountNumber().compareTo(to.getAccountNumber()) < 0) ?
from : to;
            Account second = (from.getAccountNumber().compareTo(to.getAccountNumber()) < 0) ? to
: from;

            synchronized (first) {
                synchronized (second) {
                    if (from.withdraw(amount)) {
                        to.deposit(amount);
                        System.out.println("  [TRANSFER SUCCESS] Rs. " + amount + " transferred
from " + from.getAccountNumber() + " -> " + to.getAccountNumber());
                    } else {
                        System.out.println("  [TRANSFER REJECTED] Insufficient balance in " +
from.getAccountNumber());
                    }
                }
            }
        }
    }

    public static void main(String[] args) {
        System.out.println("RUDRA MODI || 25DCS058");
        System.out.println();
        System.out.println("Practical 10 - Part B: MiniBank Concurrent Transaction Processor
Engine");
        System.out.println();
        Account accA = new Account("AC1001", "Rudra Modi", 10000);
        Account accB = new Account("AC1002", "Dhyey Patel", 8000);

        System.out.println("Initial State:\n  " + accA + "\n  " + accB + "\n");

        TransactionProcessor processor = new TransactionProcessor(4);

        System.out.println("Submitting Batch of Concurrent Cross-Transfers to Managed Thread
Pool:");

        processor.submit(() -> TransactionProcessor.transferDeadlockFree(accA, accB, 1500));
        processor.submit(() -> TransactionProcessor.transferDeadlockFree(accB, accA, 500));
        processor.submit(() -> TransactionProcessor.transferDeadlockFree(accA, accB, 2000));
        processor.submit(() -> TransactionProcessor.transferDeadlockFree(accB, accA, 1000));

        processor.stop();
```

```
        System.out.println("\nBatch Processing Complete. Transaction Processor Stopped.");
        System.out.println("Final State:\n  " + accA + "\n  " + accB);
    }
}
```

Output:

```
PS E:\B.Tech CSE CHARUSAT\SEM 3\OOP\Practicals> java Presentation\Sem-3.OOP.Prac-10\PartB.java
RUDRA MODI || 25DCS058

Practical 10 - Part B: MiniBank Concurrent Transaction Processor Engine

Initial State:
  [AC1001 | Rudra Modi | Balance: Rs. 10000]
  [AC1002 | Dhyey Patel | Balance: Rs. 8000]

Submitting Batch of Concurrent Cross-Transfers to Managed Thread Pool:
  [TRANSFER SUCCESS] Rs. 1500 transferred from AC1001 -> AC1002
  [TRANSFER SUCCESS] Rs. 500 transferred from AC1002 -> AC1001
  [TRANSFER SUCCESS] Rs. 2000 transferred from AC1001 -> AC1002
  [TRANSFER SUCCESS] Rs. 1000 transferred from AC1002 -> AC1001

Batch Processing Complete. Transaction Processor Stopped.
Final State:
  [AC1001 | Rudra Modi | Balance: Rs. 8000]
  [AC1002 | Dhyey Patel | Balance: Rs. 10000]
PS E:\B.Tech CSE CHARUSAT\SEM 3\OOP\Practicals>
```

Key Questions & MiniBank Architecture Analysis:

Q1: How does TransactionProcessor prevent deadlocks during inter-account transfers?

Answer: TransactionProcessor compares the two account numbers (e.g., fromAcc.getAccountNumber().compareTo(toAcc.getAccountNumber())) and always acquires locks in consistent ascending order, eliminating circular wait conditions.

Q2: What advantages does ExecutorService provide for MiniBank batch processing?

Answer: ExecutorService manages a fixed pool of reusable worker threads, preventing thread creation overhead, controlling concurrent load, and providing thread-safe batch transaction processing with graceful shutdown.